

The Islamia University of Bahawalpur
Department of Information Technology
Faculty of Computing, Baghdad-ul-Jadeed Campus, Bahawalpur



The Islamia University of Bahawalpur

Final Year Project Proposal

Project Title: Design and Development of a Free, User Controlled AI Browser Agent with Plan Mode and Evidence Linked Task Execution

Nature of Project: ☑ Development ☐ Research

Area of Specialization:

☑ Artificial Intelligence (AI)	☐ Game Development	☐ Internet of Things (IoT)
☐ Machine Learning	☐ Mobile App Development	☐ Blockchain
☐ Data Science and Analytics	☑ Web Development	☐ Cybersecurity
☐ Image Processing	☐ Desktop Application	☑ Natural Language Processing (NLP)

☐ Other (Specify) _____

Introduction:

Students, researchers, and working professionals spend hours on web work that repeats itself. They open the same pages, copy facts into notes, and compare details across tabs. A browser agent should take that work in plain words and carry it out in the browser the user already uses.

Frontier labs have moved in this direction with paid browser agents. Anthropic offers Claude in Chrome on paid Claude plans. It reads the page from a side panel and takes actions such as clicking links, typing text, and filling out forms, using existing logins.

OpenAI offered a similar agent mode in ChatGPT Atlas for paid subscribers. These systems show that the work is real and that users want it. They also leave a clear gap for students who cannot pay for a plan.

This project proposes a free, user controlled AI browser agent built from scratch by the student. The agent runs inside the browser, in a side panel beside the page. The user writes a task in plain words, reviews the plan first, and then watches each step run with proof.

Plan Mode shows the plan as numbered steps before any action runs. The user can read, edit, approve, or stop it. Task Mode then carries out the approved steps and links every result to its source page and quoted text.

The user keeps control at all times. Provider choice, model choice, site scope, approval for sensitive actions, and Stop stay with the user. The agent itself carries no subscription, and

the system shows an estimate of model cost before a run.

Problem Statement:

Paid browser agents solve part of the problem. They leave limits that shape this project:

- Paid access only: Claude in Chrome reached general availability on paid Claude plans. At launch, access was limited to a small group of Max subscribers on plans priced at 100 to 200 dollars per month.
- Closed provider choice: The paid agents do not support free use or a user supplied API key in the same way. The user cannot pick a cheaper model, a local model, or a provider already in use for other work.
- Weak data control: Page content is sent to the provider server that runs the agent. The user has limited say over where page text goes and what is kept.
- No clear plan to check first: An agent that acts step by step without showing its plan, its evidence, and its approval points forces the user to recheck everything after the fact.
- Results are hard to reuse: Answers given as chat text, with no source link for each fact and no saved task to run again, must be copied and rebuilt by hand each time.

Therefore, the proposed system will provide a free browser agent with Plan Mode, step by step execution, evidence linked results, approval for sensitive actions, provider and local model choice, and saved tasks.

Project Goal / Objectives:

The main goal of this project is to develop a free, secure, and user controlled AI browser agent. It will simplify web based research, comparison, form help, and routine page work for students and researchers.

Users will write tasks in plain words, review a clear plan before execution, and receive results that link each fact to its source page. The system will use TypeScript, React, and Tailwind CSS, with a browser control layer built from scratch.

1. Build a planner, executor, and verifier in one agent loop, with Plan Mode, step approval, Stop, and an audit log in plain words.
2. Add an evidence layer so every extracted fact carries a source URL, quoted text, and capture time. Facts with no source must be marked as missing, not guessed.
3. Build a generative task interface with evidence cards, comparison tables, and approval forms drawn only from a registered component set, with props checked before render.
4. Support user supplied model keys across major providers and local models, show a cost estimate before each run, and provide a saved task library with rerun and export.

Application Architecture / Technology Stack / Tools / Hardware:

The system will use a local, in browser architecture. The side panel will provide the user view and task controls. The agent core will provide planning, execution, and verification.

The browser control layer will read the page, act on page elements, and report what really happened after each action. No central agent server is used. Keys stay in the browser and go only to the model provider the user picks.

1. Frontend and interface: React, TypeScript, Tailwind CSS, HTML5, CSS3, JavaScript. Side panel with Plan Mode, Task Mode, evidence cards, comparison tables, approval forms, and task library.

2. Agent core, built from scratch: Planner, Executor, and Verifier. Plan Mode, step approval, Stop, and audit log in plain words.

3. Browser control layer, built from scratch: page reading, element identification tied to the exact element, visibility and enabled checks before action, click, type, scroll, and tab navigation. Built on Chrome DevTools Protocol and browser automation APIs.

4. Generative task interface: registered component set for result cards, tables, forms, and evidence views. Model output is checked against a strict schema and only registered components are drawn.

5. Models and providers: user supplied keys for OpenAI, Anthropic, Google Gemini, OpenRouter, DeepSeek, Mistral, and OpenAI compatible endpoints. Local models through Ollama and LM Studio. Each model must support tool calling.

6. Storage and privacy: local browser storage for keys, tasks, history, and audit log. No browsing data is sent to a central agent server. Site scope, Allow, Ask, and Block rules are set by the user.

7. Tools: Visual Studio Code, Git, GitHub, browser developer tools, Node.js, package manager, and unit test tools.

8. Hardware: Laptop or desktop with a modern multi-core processor, 8 GB RAM minimum (16 GB recommended), stable internet for model API calls, and a current web browser.

Scope, Feasibility and Evaluation

Scope

The scope includes plain word task entry, Plan Mode, step by step execution, evidence linked extraction, comparison, summarising, and form help. It also includes saved tasks, approval for sensitive actions, site scope rules, and export of results with sources.

Sensitive steps such as submit, send, delete, upload, pay, password entry, and code entry are handed to the user or held for approval. They are never run without a clear user decision.

Feasibility

The project is technically and economically feasible. It will use standard web and browser technologies, free development tools, and user supplied or local models. The agent itself carries no subscription.

Its modular design separates interface, agent core, browser control, and verification. This

supports step by step building and later improvement within two semesters for a single student.

Evaluation

The system will be checked through functional checks, usability checks, and accuracy checks. Evaluation will focus on task completion, correctness of facts against ground truth, evidence link accuracy, and approval behaviour on sensitive steps.

Results will be recorded task by task, with errors and failed steps kept in the record. This will show the strengths and limits of the system plainly.

8.1 Evaluation Criteria

- Functional checks: verify task entry, Plan Mode, step approval, Stop, site scope rules, saved tasks, rerun, and export with sources.
- Accuracy checks: verify extracted facts against ground truth pages. Record facts with missing or wrong sources as errors.
- Control checks: verify that sensitive actions pause for approval, that refused steps do not run, and that the audit log records actions, approvals, refusals, and errors.
- Usability checks: conduct task based walkthroughs for student users. Record time to first successful task, setup errors, and issues for improvement.
- Cost and time checks: record time and model cost per task. Compare results across providers and local models where available.

9. Expected Outcomes and Deliverables

- A free AI browser agent that runs inside the browser, with Plan Mode, Task Mode, evidence linked results, and user control.
- Plain word task entry, step review and approval, step by step execution, and Stop at any time.
- Evidence cards and comparison tables with source links, quoted text, and capture time for each fact.
- Saved task library, rerun with new inputs, and export of results with sources.
- Provider and local model choice, cost estimate before each run, site scope rules, and audit log.
- Architecture and design documentation, task set with ground truth, evaluation results, final report, and project demonstration.

10. Novelty and Significance

The project does not claim that browser agents, large language models, side panels, or plan views are individually new. Its contribution is the integration of these parts in a single student built system.

That integration is a free agent with no subscription, provider and local model choice, Plan

Mode with step approval, evidence linked results, approval and audit control, and saved reusable tasks. It serves students who cannot use paid browser agents, and it gives proof and control that chat style answers do not provide on their own.

Student Data:

Sr.	Reg. No.	Name	Phone	Signature
1	F23BIBF1M01069	Jahangir Iqbal	03023684057	

References:

1. Anthropic. Claude in Chrome: AI agent for the Chrome browser. Reported launch, pilot limited to Max subscribers. https://en.ain.ua/2025/08/28/anthropic-has-introduced-claude-for-chrome/

2. Anthropic. Claude in Chrome, general availability on paid Claude plans, 26 August 2026. Reported in AI browser coverage, October 2026. https://ai.blocksize.hr/blog/en/ai-browsers-agentic-browsing-2026.html

3. OpenAI. ChatGPT Atlas, AI powered browser with sidebar assistance and agent mode for paid subscribers. https://yourstory.com/ai-story/openai-takes-on-google-perplexity-in-ai-browser-ra

4. OpenUI. Docs: build Generative UI from your own components. OpenUI Lang, component library, and streaming renderer. https://www.openui.com/docs

5. Meta. React Documentation. https://react.dev/

6. Tailwind Labs. Tailwind CSS Documentation. https://tailwindcss.com/docs

7. The Islamia University of Bahawalpur. Baghdad-ul-Jadeed Campus page. Main campus on Hasilpur Road, Bahawalpur. https://www.iub.edu.pk/baghdad-ul-jadeed-campus

Project Supervisor:

Mr. Munawar Ahmad

Supervisor Signature: _____